

TechStore DS

Marketplace académico de productos tecnológicos con visualizadores de estructuras de datos

Integrantes: Daniel Vivas, Danie Ruiz

Deploy: techtstore.netlify.app

Repositorio: <https://github.com/Daniielee/Proyecto-ED2.git>

Resumen

TechStore DS es una aplicación frontend educativa y demostrativa que combina un pequeño marketplace (catálogo y detalle de productos) con implementaciones y visualizaciones de estructuras de datos clásicas (LinkedList, Stack, Queue, Trie, Heap y Graph). El objetivo del proyecto es integrar conceptos de Estructuras de Datos y Algoritmos en una aplicación React + TypeScript con interacción real vía Firebase.

Tecnologías Principales

Categoría	Tecnología
Frontend	React 19 + TypeScript + Vite
Estilos	SCSS Modules (sintaxis @use) y mixins compartidos
Autenticación y BD	Firebase (Authentication, Firestore, Realtime Database)
Deploy	Netlify (frontend)
Control de versiones	Git + GitHub

Objetivo y Alcance

TechStore DS provee las siguientes funcionalidades:

- Catálogo de productos con búsqueda predictiva (Trie).
- Páginas de detalle con historial de productos visitados (LinkedList).
- Ranking de top productos usando un Max-Heap.
- Historial de navegación (Stack) y visualizaciones académicas.
- Chat en tiempo real implementado con una Queue y Firebase Realtime Database.
- Autenticación real con Firebase Auth y rutas privadas protegidas.
- Visualizadores interactivos para Heap y Graph en secciones de detalle de productos.

Limitaciones Conocidas

- Lógica principalmente client-side (sin backend propio fuera de Firebase).
- Historial en memoria — no persistente entre sesiones, salvo datos guardados en Firestore/RTDB.
- El chat carga todos los mensajes al abrir (no paginado).

Estructuras de Datos Implementadas

A continuación, se describe cada estructura, su uso en el proyecto y la motivación técnica con complejidades en notación Big-O.

Estructura	Uso en el proyecto	Complejidad clave
LinkedList	Historial de productos visitados en detalle (tope 5 elementos)	append/prepend $O(1)$, remove tail $O(1)$
Stack	Historial de navegación del usuario — funcionalidad tipo undo/atrás	push/pop/peek $O(1)$
Queue	Buffer de mensajes en chat antes/durante sincronización con Firebase RTDB	enqueue/dequeue $O(1)$
Trie	Búsqueda predictiva/autocomplete en la página de Products	insert/búsqueda de prefijo $O(m)$
Heap (Max)	Ranking de Top Productos por rating/preferencia (useTopProducts)	insert $O(\log n)$, extractMax $O(\log n)$, peek $O(1)$
Graph	Sistema de recomendaciones en ProductDetail — relaciones entre productos/categorías	BFS/DFS $O(V + E)$

Detalle por Estructura

LinkedList — Lista enlazada simple

- Uso: historial de productos visitados en detalle del producto (tope 5 elementos).
- Motivación: permite inserción al frente y eliminación del final en $O(1)$ con punteros adecuados. Modelo apropiado para mantener un buffer pequeño de recientes.
- Operaciones relevantes: append $O(1)$, prepend $O(1)$, remove tail $O(1)/O(n)$ según implementación.

Stack — Pila

- Uso: historial de navegación del usuario (contexto `NavigationContext`) para funcionalidades tipo undo/atrás.
- Motivación: LIFO corresponde al comportamiento del botón Atrás — la última página visitada es la primera en retornar.
- Complejidad: push/pop $O(1)$, peek $O(1)$.

Queue — Cola FIFO

- Uso: buffer de mensajes en el chat en tiempo real antes y durante la sincronización con Firebase Realtime Database.
- Motivación: mantener orden cronológico de mensajes (primero en entrar, primero en salir).
- Complejidad: enqueue $O(1)$, dequeue $O(1)$.

Trie — Árbol de prefijos

- Uso: búsqueda predictiva/autocomplete en la página de Products.
- Motivación: búsquedas por prefijo en $O(m)$ donde m es la longitud del prefijo, independiente del número total de palabras — ventajoso frente a filtros lineales sobre arrays.
- Complejidad: inserción $O(m)$, búsqueda de prefijo $O(m)$. Memoria dependiente del alfabeto y prefijos compartidos.

Heap — Max-Heap

- Uso: ranking de Top Productos (`useTopProducts`) para mantener top-k por rating/preferencia.
- Motivación: extracción del máximo en $O(\log n)$ y mantenimiento eficiente del top-k.
- Complejidad: insert $O(\log n)$, extractMax $O(\log n)$, peek $O(1)$.

Graph — Grafo no dirigido ponderado

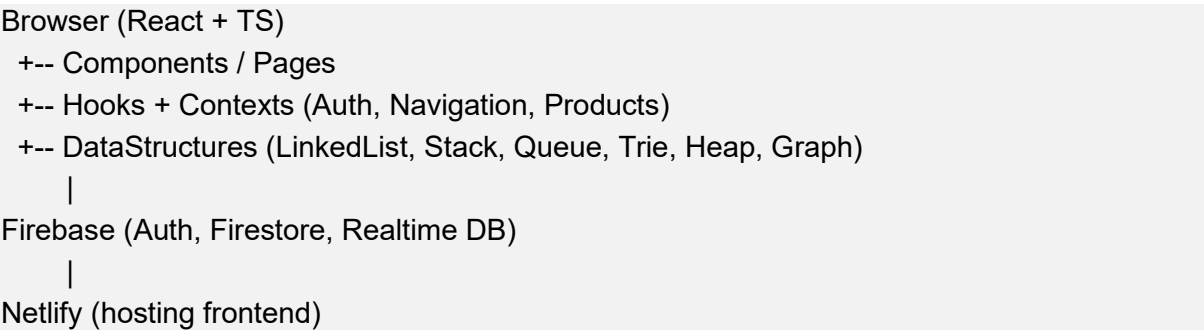
- Uso: sistema de recomendaciones en ProductDetail — relaciones por categoría/atributos entre productos.
- Motivación: modelar relaciones entre nodos para aplicar BFS/DFS o algoritmos avanzados en el futuro (Dijkstra, clustering colaborativo).
- Complejidad: BFS/DFS $O(V + E)$.

Arquitectura

La aplicación se organiza en tres capas principales:

- Capa de presentación: componentes React (páginas, visualizadores, formularios).
- Estado y lógica cliente: hooks y contexts (`useProducts`, `useAuth`, `NavigationContext`).
- Persistencia y realtime: Firebase — Firestore para CRUD persistente; Realtime Database para chat de baja latencia.
- Despliegue: Netlify sirve los assets generados por Vite.

Diagrama Simplificado



Estructura del Repositorio

```
src/
+-- components/
|   +-- layout/      # Navbar, Footer (responsive, hamburguesa)
|   +-- common/      # Visualizadores y componentes reutilizables
+-- pages/           # Vistas publicas y privadas
+-- context/         # AuthContext, NavigationContext
+-- hooks/           # useAuth, useProducts, useSearch, useChat...
+-- dataStructures/  # LinkedList, Stack, Queue, Trie, Heap, Graph
+-- firebase/        # firebase.ts (configuracion de servicios)
+-- styles/          # variables, mixins, global.scss
+-- types/           # definiciones TypeScript
```

Retos Técnicos y Soluciones

Reto	Solución aplicada
Migración Sass @import → @use	Actualización de módulos y creación de alias de variables para compatibilidad. Resultado: eliminación de errores en tiempo de compilación.
Conflictos visuales (contraste y superposición)	Ajuste global de variables en src/styles/_variables.scss y uso de mixins (card, btn-primary) para consistencia.
Permisos Firebase	Permisos temporales en Realtime DB durante desarrollo; limpieza de colecciones duplicadas manualmente en consola.
Visualización de estructuras	Se añadieron visualizadores para Heap y Graph en src/components/common. Se integraron en DataStructuresDemo usando useRef para instancias persistentes.